

LAB: Digital Image Processing: Date 23 September 2025

Dr. Irfan Ali

Assistant Professor & Coordinator Multimedia & Gaming

Hands-On Lab: Processing Video Frames using OpenCV

Video Processing

- ❖ Video = sequence of images (frames) arranged in time.
- ❖ DIP extends image processing techniques to handle **temporal data**.
- ❖ Goal: Enhance, analyze, and extract useful information from video streams.
- ❖ **OpenCV**: Widely used for real-time video/image processing.

Hands-On Lab: Processing Video Frames using OpenCV

Temporal Data

•Definition:

Temporal data is data that is related to **time**. It not only describes the values of a signal or an image but also how those values **change over time**.

•In Image/Video Processing:

- An **image** = spatial data (pixels in 2D space).
- A **video** = sequence of images (frames) changing with time → this is **temporal data**.

•Example:

- A static photo of a person → spatial data.
- A video of the person walking → spatial + **temporal data** (motion, sequence, speed).

• Importance of Processing Video Frames

- Video = sequence of images (frames).
- Allows **real-time analysis** of motion, objects, and scenes.
- Used to **enhance quality**, detect noise, and highlight features.
- Helps in **automation, surveillance, and monitoring systems**.
- Bridges the gap between **theory (algorithms)** and **practical applications (CCTV, robotics)**.

• **Applications of Video Processing**

- **Surveillance & Security:** Motion detection, face recognition.
- **Healthcare:** Medical video analysis (endoscopy, surgery).
- **Traffic Monitoring:** Vehicle detection, number plate recognition.
- **Robotics & Drones:** Object tracking, navigation.
- **Entertainment:** Video effects, filters in apps.
- **Sports Analysis:** Motion tracking, replay enhancements.

- **Main OpenCV Functions for Video Processing**

- **Video Capture:**

- `cv2.VideoCapture()` → load video or webcam.
- `.read()` → read frames sequentially.

- **Display & Control:**

- `cv2.imshow()` → display frames.
- `cv2.waitKey()` → control playback & keyboard input.

- **Frame Processing:**

- `cv2.resize()` → resize frames.
- `cv2.cvtColor()` → convert color spaces (RGB ↔ Gray).
- `cv2.GaussianBlur()`, `cv2.Laplacian()`, `cv2.Sobel()` → filters.

- **Resource Management:**

- `cap.release()` and `cv2.destroyAllWindows()` → clean exit.

- **Functions elaboration**

1. **Load Video** → `cv2.VideoCapture()`.
2. **Read Frames in Loop** → `cap.read()`.
3. **Resize & Process Frames** → `filters, color conversion.`
4. **Stack Frames (Optional)** → `np.hstack(), np.vstack()`.
5. **Display Results** → `cv2.imshow()`.
6. **Exit with Key Press** → `cv2.waitKey()`.

Hands-On Lab: Processing Video Frames using OpenCV

Opens the video file for reading.

```
cap = cv2.VideoCapture("robot.mp4").
```

```
[ ]: while cap.isOpened():  
      r, frame = cap.read()
```

cap.read() returns two values:

r (True/False): Success flag.

frame: The captured frame.

Hands-On Lab: Processing Video Frames using OpenCV

Display Frame

```
[ ]: # Resize image , Resizes each frame to 600 x 500 pixels.  
frame = cv2.resize(frame, (600, 500))
```

Exit Condition

```
[ ]: # Waits 30 ms between frames. Stops playback when user presses 'p'.  
  
if cv2.waitKey(30) & 0xff == ord("p"):  
    break
```

Hands-On Lab: Processing Video Frames using OpenCV

Release Resources

```
[ ]: # Frees video resource and closes all windows.  
cap.release()  
cv2.destroyAllWindows()
```

- Run the program step by step.
- Observe how frames are displayed.
- Press 'p' to stop the video.

Hands-On Lab: Processing Video Frames using OpenCV

```
•[1]: import cv2
      cap = cv2.VideoCapture(r"C:\Users\HP\Desktop\DIP LAB/robot.mp4")

      while cap.isOpened():
          r, frame = cap.read()
          if r == True:
              frame = cv2.resize(frame, (600, 500))
              cv2.imshow("Ali", frame)
              if cv2.waitKey(30) & 0xff == ord("p"):
                  break
          else:
              break

      cap.release()
      cv2.destroyAllWindows()
```

Task: Apply hstack/vstack and LP filter, Perform this task and change parameters for practicing

```
[3]: import cv2
import numpy as np

# Load video
cap = cv2.VideoCapture(r"C:\Users\HP\Desktop\DIP LAB/robot.mp4")

while cap.isOpened():
    r, frame = cap.read()
    if r:
        # Resize frame
        frame = cv2.resize(frame, (300, 250)) # smaller size for stacking

        # Convert frame to grayscale (just for variety)
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        gray = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR) # convert back to 3 channels for stacking

        # Apply Gaussian blur
        blur = cv2.GaussianBlur(frame, (7, 7), 0)
```

Task: Apply hstack/vstack and LP filter, Perform this task and change parameters for practicing

```
# Show results
cv2.imshow("Original + Gray + Blur (HStack)", hstack)
cv2.imshow("Original + Blur (VStack)", vstack)

# Stop when 'p' is pressed
if cv2.waitKey(30) & 0xFF == ord("p"):
    break
else:
    break

cap.release()
cv2.destroyAllWindows()
```

Task: Apply High pass filter & its types, Perform this task and change parameters for practicing

```
[2]: import cv2
import numpy as np

# Load video
cap = cv2.VideoCapture(r"C:\Users\HP\Desktop\DIP LAB/robot.mp4")

while cap.isOpened():
    ret, frame = cap.read()
    if ret:
        # Resize frame
        frame = cv2.resize(frame, (600, 500))

        # ----- Low Pass Filter (Smoothing) -----
        low_pass = cv2.GaussianBlur(frame, (7,7), 0)

        # ----- High Pass Filter (Edge detection using Laplacian) -----
        gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
        laplacian = cv2.Laplacian(gray, cv2.CV_64F)
        laplacian = cv2.convertScaleAbs(laplacian)
```

Task: Apply High pass filter & its types, Perform this task and change parameters for practicing

```
# Show windows
cv2.imshow("Original", frame)
cv2.imshow("Low Pass (Gaussian Blur)", low_pass)
cv2.imshow("High Pass (Laplacian)", laplacian)
cv2.imshow("High Pass (Sobel)", sobel)

# Press 'p' to stop
if cv2.waitKey(30) & 0xFF == ord('p'):
    break
else:
    break

cap.release()
cv2.destroyAllWindows()
```